Akshmen Sutharshan - 20576162

System Manual

Software Description

(Maximum 1 page)

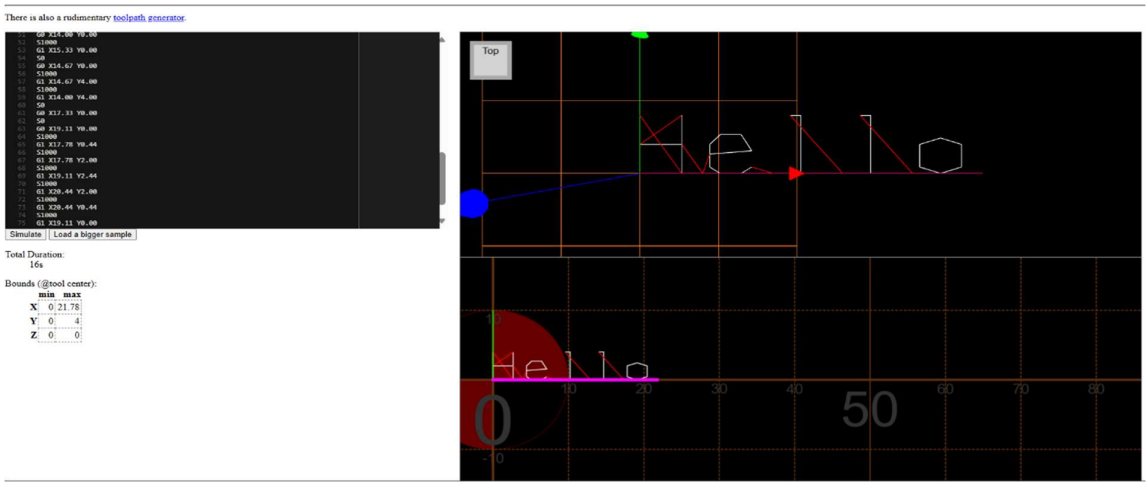
This software purpose is to monitor and control a drawing robot through serial communication (RS232) by converting a user inputted text file containing words into G code commands to draw specific characters.

The program first checks and confirms the serial connection to the Arduino. Then the SingleStrokeFont.txt file is parsed by the LoadSSFfile function [F1]. The SSFfont structure array is populated by [F1], mapping the ASCII characters to the movement commands specific to them. The program prompts to user to input a desired text height through the terminal and checks if the input is valid between 4 and 10 mm inclusive [F2]. The ScaleFactorCalculation function [F3], is used to divide the users inputted value by the grid size (18), allowing the generated Gcode commands to be scaled to the requested size.

Next, the text.txt file is read word by word by the [F4] function. Within this loop function, each width of the word that is to be drawn next is pre calculated by [F12] to ensure that the word does not exceed the 100mm maximum line width. If it does however, [F4] calls [F8] to initiate a line break. The [F8] function resets the X position of the pen back to left hand margin and moves the Y position of the pen 5 mm down.

The [F5] function is then called to retrieve the movement data for each character contained in the word using the SSFfont structure arrays. [F6] then handles the conversion of coordinate data from ASCII characters into G code strings using the scale factor that was calculated by [F3]. The program understands the PEN state commands, (S0 =PEN is up) and (S1000 = PEN is down), as well as movement commands (G0 = rapid movement, G1 = Slow linear drawing movement). Although the font file automatically monitors character offsets, I added an extra offset of 2 units (will be scaled) after each character to ensure the words are not too close together. 5 units (will be scaled) between each WORD is added to prevent characters overlapping each other. [F7] is then called to update the X and Y coordinates of the pen after every character and word drawn based on these offsets.

Before the robot begins to draw, [F9] is called to start up the robot by using commands S0 and M3 to raise the pen and power the motors respectively. An implemented function, PrintBuffer, transmits all the Gcodes to the Arduino. The system then waits for a signal from [F10] to ensure the software and physical robot are synchronised. After the text file has been processed fully, [F11] function is called to raise the pen and move the robot back to its original position. The drawing is now completed.



Project Files

(Maximum 1 page)

main.c

This is where program starts and manages the flow by calling each function needed sequentially. This involves starting up the robot, process the text file using the font file, generate G codes. SerialPort (declared as 4) is defined here to track the COM port used, but the actual configuration for the COM port connection is defined in serial.h .

PenDrawing.h

This is the header file for the logic needed for the drawing robot. The structures (MC and SSFfont) are defined here, and the file contains the function prototypes that are used between PenDrawing.c and main.c

PenDrawing.c

This C file contains the logic created for the drawing robot. The 12 functions here contain the logic such as: parsing the SingleStrokeFont.txt file, ask and validate the users desired text height, calculate the float ScaleFactor to scale G code commands, update the X and Y coordinates of the pen to ensure offsets between words. Most importantly, [F4] carries out the main loop that reads the text.txt file to generate and finalise G code strings.

serial.h

This is the header file for the (rs232) serial communication library. Function that are declared here are used for opening the port, ensure the data buffers are sent and then waiting for “ok” responses from the Arduino. The COM port is set here through the #define cport_nr and the baud rate is set through #define bdrate. These parameters should be updated to match the Arduino that is connected.

serial.c

This C file contains the logic for Serial communication (rs232). There is a preprocessor directive #define Serial_Mode, which compiles for the robot when left uncommented. If it is commented, the code compiles and send G codes to the terminal which can be moved to the G code simulator.

rs232.h

The header file for rs232 library

rs232.c

This C file contains the library logic that handles the serial port communication for Windows

SingleStrokeFont.txt

This text file contains the coordinate data for the ASCII characters. The program will parse, scale and convert the data into G code strings. This is sent to the robot.

test.txt

This text file contains a sample of text inputted by the user to be drawn by the robot. This file is read by [F4] which sequentially calls other functions to ensure the resulting G code cause the robot to draw the text to a high precision.

Key Data Items

Name	Data type	Rationale
FontCharacterList	Array of Structs SSFfont [128]	This array of structures will store data for each 128 ASCII characters from the SingleStrokeFont.txt file. By using an array, each character can be accessed easily using its specific ASCII code. Structs was used because there are different types of data (NewMarker, ASCIIcode, NumberMoves) it needs to hold. This supports [F1] with loading the font and [F5] for accessing the movement commands of a specific character.
Height	Float	This variable will hold the input value requested by the user, between 4 to 10 mm inclusive. A float variable contains decimal positions, which can provide precision. This data is gained through [F2].
ScaleFactor	Float	This variable will hold the value after the "height/18" calculation. Dividing the user input by 18 will result in a decimal value. A float is used so this value does not round down to 0 since that would mean the robot would not draw. This value is returned by [F3] and is used by [F6] to scale the coordinates of G code commands.
Word	Char Array [100]	This array will store the word that is being read from the file. I used 100 because there should not be many words that exceed 100 characters. [F12] uses this to calculate the width of the word before it is drawn.
MoveCommands	Array of Structs MC [35]	This array of structures will store the sequence of Movement Commands needed for the current character being drawn. A struct is used because each movement commands holds data (PEN state, X, Y). A size of 35 is used because the most complex character to be drawn requires less than 35 moves. This is used in [F5] and is used by [F6] to convert the commands into Gcode.
Gcodebuf	Char Array [50]	This Char array will contain strings for a single G code command which the robot will execute (e.g. = G0 X0 Y0 S1000). An array size of 50 will be able to store to longest possible G code command to prevent a overflow of memory stored. This is used in [F6].
Xcurrent Ycurrent	Float	These floats track the CURRENT (X, Y) coordinates of the pens drawing position. Float was used to provide decimal precision for the G codes. These variables are passed by reference to [F6, F7, F8]. This ensures that the PEN position is updated to all functions.
LineBreakSpace	Const Float	This constant float stores 5.0 which is the value for each line break spacing. The Y coordinate will drop down by 5 units after a line break is triggered. Const Float was used because the float value must stay constant since it was requested by the coursework. This is used by [F8] when a line break occurs.

MaxLineWidth	Const Float	This constant float stores 100.0 (mm) due to the maximum line width limit. This is used by [F4] when it calls [F8] to ensure the width of the word will not overlap then max width of 100mm and will instead trigger a line break.
PENstate	Int	This variable tracks between pen states (0 = Pen raised, 1 = Pen down). Pointers pass this variable to [F9, F11] to set the pen state up (0) when starting up and shutting down the robot. This will prevent accidental drawing during these stages.
okBuf	Char Array [16]	This buffer stores “ok” responses by the Arduino during serial communication. This buffer is used by [F10] and used locally by [F6, F8, F9, F11] to manage “ok” response while checking for error messages. This ensures smooth serial transmission.

Extend table as required

Functions

Only include functions that you have written yourself.

Only include functions that you will develop.

Inputs, Outputs will be declared as (i) and (o) respectively

[F1] = LoadSSFfile (const char* Filename, SSFfont FontCharacterList [128])

Parameters:

- (i) Filename = SingleStrokeFont.txt file path
- (o) FontCharacterList = This array will store SSFfile font structures (ASCII 0-127)

Return value = If this Is successful then this will return 128 and if it fails then return -1

[F2] = UserInputTextHeight (void)

Parameters:

- No input or output parameters

Return value = If successful, a float measurement between 4 and 10 mm will be returned. However, it will return -1 to notify the user that there was an error and is prompted to put a number within the range provided.

[F3] = ScaleFactorCalculation (float Height)

Parameters:

- (i) Height = This is the variable that stores the number the user inputted
- There are no output parameters

Return value = This will return the result from (Height/18) as a float

[F4] = ProcessTheTextFile (const char *TextFile, SSFfont FontCharacterList [] , float ScaleFactor)

Parameters:

- (i) TextFile = This is the file path string containing characters from the text file to be drawn
- (i) FontCharacterList = This array will store font data (ASCII 0-127) and is used by [F5] to lookup movement data for characters
- (i) ScaleFactor = This variable is used by F6 to scale G code commands

Return value = If a file error (file is not opened) occurs then -1 will be returned. If a word is found that is wider that the max line width (100) then this will return -2. 0 will be returned if all the characters in the text file is processed.

[F5] = CharacterMovementsNeeded (char CharCurrent, SSFfont FontCharacterList [], MC MoveCommands [])

Parameters:

- (i) CharCurrent = will store the current ASCII character from the text file that will be processed
- (i) FontCharacterList = This array is used to lookup movement data for font characters
- (o) MoveCommands = This array stores Movement Commands for the current CharCurrent to be converted to G code commands

Return value = If the character is valid, the number of movements is stored in MoveCommands then it will return the number of movements. If a character is not found, then it will return -1. If 0 is returned, then there is a “space”, and no movements are added to the array.

[F6] MovementToGcode (MC MoveCommands [], int NumOfMovement, float *Xcurrent, float *Ycurrent, float ScaleFactor, char *Gcodebuf, int SerialPort)

Parameters:

- (i) MoveCommands = This array stores Movement Command structures for the character that was processed
- (i) NumofMovement = This integer contains the number of movements in the MoveCommands array
- (i, o) Xcurrent/Ycurrent = The pointers will read the starting position of the pen and update the X, Y position of the pen after drawing the character
- (o) Gcodebuf = This buffer will format and contain the Gcode strings.
- (i) ScaleFactor = Used to scale the coordinates of the G code
- (i) SerialPort = This contains the integer that identifies the port index for the transmission context. This is passed to [F10]

Return value = If the G code string is generated and stored inside Gcodebuf then this will return 0. If the coordinates are invalid after scaling this will return -1

[F7] CharacterPositionUpdate (float *Xcurrent, float *Ycurrent, float Xfinal, float Yfinal)

Parameters:

- (i, o) Xcurrent/Ycurrent = These pointers will read the current position and update the positions to the equal Xfinal and Yfinal . This then becomes the new starting position for the character to come.
- (i) Xfinal/Yfinal = These floats will update the X,Y position to the last movement of the character that was just drawn

Return value = 0

[F8] LineBreakHandling (float *Xcurrent, float *Ycurrent, float MaxLineWidth, float LineBreakSpacing)

Parameters:

- (o) Xcurrent = This pointer reads the current X position of the pen. X then changes to 0.
- (i, o) Ycurrent = This pointer reads the current Y position of the pen. Y decrements by 5 (LineBreakSpacing variable is used)
- (i) MaxLineWidth = checks the current line width against the defined maximum limit (100mm)
- (i) LineBreakSpacing = this variable is equal to 5 which is the distance to move down from one completed line to the start of a new line

Return value = 0

[F9] RobotStartUp (int SerialPort, int *PENstate, char Gcodebuf[])

Parameters:

- (i) SerialPort = This is passed to [F10] for flow control
- (o) PENstate = This pointer will keep track of the pen state, 0 is raised and 1 is drawing state. This will be set to 0 initially.
- (o) Gcodebuf = This function uses this buffer to temporarily hold start up G code commands before it is sent(e.g. M3, S0)

Return value = 0, if all commands to initialise the robot was created and carried out successfully. It will return -1 if any command was not sent successfully.

[F10] GcodetoArduino (const char *Gcodebuf, int SerialPort, char okBuf [])

- (i) Gcodebuf = contains G code string to send to Arduino
- (i) SerialPort = This contains the integer that identifies the port index for the transmission context
- (o) okBuf = This buffer stores Arduino responses (“ok”)

Return value = If “ok” is received then this will return 0 and -1 if no “ok” response is received then this will return -1

[F11] ReturnPenToOrigin (int SerialPort, float *Xcurrent, float *Ycurrent, int *PENstate, char Gcodebuf [ ])

Parameters:

- (i) SerialPort = Stores the port index used
- (o) Xcurrent/Ycurrent = The pointers will reset to (0,0) on the robot drawing interface
- (o) PENstate = This pointer will set the pen state to 0 (raised)
- (o) Gcodebuf = This function uses this buffer to temporarily contain the G code command to return the pen to the origin

Return value = 0

[F12] CalculateTheWordWidth (const char *WordBufer, SSFfont FontCharacterList [], float ScaleFactor)

Parameters:

- (i) WordBufer = The buffer contains the string of the word that is to be measured
- (i) FontCharacterList = This array contains the font data
- (i) ScaleFactor = This scaling variable will ensure that the calculation matches the output

Return value = If successful, the width of the word will be returned as a float. This will assist the main loop to check if the word will exceed the maximum line width limit before drawing.

Testing Information

success means program continues, fail means there is an error in the program

Function	Test Case	Test Data	Expected Output
[F1] LoadSSFfile	(1) File loaded	Filename = SingleStrokeFont.txt	Returns 128 (success)
	(2) File does not exist	Filename = NotSingleStrokeFont.txt	Return -1 (fail)
[F2] UserInputTextHeight	(1) Valid number within range	UserInput = 5	Returns 5.0 (success)
	(2) Boundary numbers	UserInput = 4 , 10	Returns 4.0 , 10.0 (success)
	(3) Invalid numbers out of range	UserInput = 15	Return -1 (fail)
	(4) Not a number	UserInput = hello, %%%	Return -1 (fail)

[F3] ScaleFactorCalculation	(1) Middle range scaling	Height = 9.0	Returns 0.50
	(2) Boundary scaling	Height = 4.0	Returns 0.22
[F4] ProcessTextFile	(1) TextFile is read	TextFile = TextFile.txt	Returns 0 (success)
	(2) TextFile does not exist	TextFile = //NotExist.txt	Return -1 (fail)
	(3) TextFile contains nothing	TextFile = Nothing.txt	Returns 0 (file opens but no characters are read, loops 0 times) (success)
	(4) TextFile contains a word that is too long	TextFile = “some very long word” in TextFile.txt	Returns -2 (error trapping) (fail)
[F5] CharacterMovements Needed	(1) Character is listed	SSFfont[128] CharCurrent = S	Returns No. of movement commands for “S” (success)
	(2) Character is not listed (invalid ASCII)	CharCurrent = (ASCII not within range 0 and 127)	Return -1 (fail)
	(3) Inputting a space between words	CharCurrent = (space)	No. of movements to input a space = 0 Return 0 (success)
[F6] MovementToGcode	(1) Movement is converted	MoveCommands = contains valid data for a letter (e.g. S). NumofMovement = (no.) of moves needed for (S.)	Returns 0 Gcodebuf contains strings for Gcode to be sent. (success)
	(2) No movements (space)	NumOfMovement = 0	Returns 0 Gcodebuf does not contain anything to be sent. (success)
	(3) SerialPort transmission fails	SerialPort = No connection	Return -1 (fail)

[F7] CharacterPosition Update	(1) Positioning of the next character is updated	Xcurrent = 5.0 Ycurrent = 5.0 Xfinal = 10.0 Yfinal = 5.0	Returns 0 Xcurrent now becomes 10.0 Ycurrent now becomes 5.0 (success)
	(2) Reset positioning to origin	Xcurrent = 95.0 Ycurrent = 5.0 Xfinal = 0.0 Yfinal = 0.0	Returns 0 Xcurrent now becomes 0.0 Ycurrent now becomes 0.0 (success)
[F8] LineBreakHandling	(1) Line width has reached or gone over 100mm	Xcurrent = 100.0 or 104.0 Ycurrent = 0.0 MaxLineWidth = 100.0 LineBreakSpacing = 5.0	Returns 0 Xcurrent resets to 0 and Ycurrent updates to negative 5.0 (success)
	(2) Multiple lines in text file. Sequential line breaks needed	Xcurrent = 95.0 Ycurrent = -5.0 MaxLineWidth = 100.0 LineBreakSpacing = 5.0	Returns 0 Xcurrent resets to 0 and Ycurrent updates to negative 10.0 (success)
[F9] RobotStartUp	(1) Robot starts up	SerialPort = (a valid connected port e.g. 4) (COM 4)	Returns 0 PENstate = 0, Gcodebuf contains startup Gcode strings (e.g. M3,S0) (success)
	(2) Robot does not start up	SerialPort = (invalid connected port e.g. -2)	Returns -1 Commands are not sent through (fail)
[F10] GcodetoArduino	(1) “ok” response confirmed after command	Example Gcodebuf G1 X5 Y5 F1000 Connection is valid to a COM port on Arduino (e.g. 4)	Returns 0 G code string is sent and “ok” response is waited for and confirmed. (success)
	(2) “ok” response is not confirmed after command	Example Gcodebuf G1 X5 Y5 (No F) Arduino does not respond	Does not return (Program waits for a ok response) (fail)

[F11] ReturnPenToOrigin	(1) Return pen to the origin (0,0)	SerialPort = check with Arduino if connection is successful Xcurrent = 30.0 Ycurrent = 20.0 PENstate = 1	Returns 0 Xcurrent = 0.0 Ycurrent = 0.0 PENstate = 0 G code moves pen back to (0,0) (success)
[F12] CalculateTheWord Width	(1) Word width confirmed and can be drawn	WordBufer = “hello” ScaleFactor = 2.0	Returns total width of the word (success)
	(2) There is no word so no word width is measured	WordBufer = nothing	Returns 0 Moves on to next word in string (success)
Main ()	(1) Successful completion of all code	SingleStrokeFont.txt Text.txt are both loaded Height is acquired by user input	Returns 0 Printf statements showing: “The font file has loaded” (….more statements) “Finished” (success)
	(2) SSFont file is not found	SingleStrokeFont.txt file is not found	Returns -1 Printf statement shows “The SingleStrokeFont.txt file was not found” [F1] fails and programs exits (fail)
	(3) TextFile is not found	The users TextFile.txt is not found	Returns 0 Printf statement shows “The text file was not found” The robot will return to origin point (fail)
	(4) Successful Line breaks	Text.txt contains a text that exceeds 100mm limit	Successful insertion of line break. Print f statement “The drawing has completed”. Robot moves Y coordinates down by 5 mm to draw the next line

Extend table as required. Note that 'Function' includes main()

AI Statement

Error Identification and Correction: The AI helped correct inconsistencies my code implementation, which helped identify and correct specific issues such as the pen state logic error in [F6].

Grammar and Clarity: The AI suggested improvements to grammar, spelling, and phrasing to make the document more professional and clearer.

Logic and Structure Refinement: While I designed the core logic, the AI helped refine the structure of function declarations and improved the completeness of the flow in flowcharts.

Flowchart(s)

Included as separate pdf